



(/rol/app/)

Help

Korean

검색

Seunghe...

Sites

홈 (/rol/app/) (home) > Securing the OpenShift Service Mesh > Chapter 1: Securing the OpenShift Service Mesh

Securing the OpenShift Service Mesh

Lesson

Lab Environment

Guided Exercise: Mutual TLS (mTLS) with OpenShift Service Mesh

Configure and validate strict mTLS enforcement at the namespace level.

Verify that service-to-service authentication by using Kiali visualization and traffic testing.

Outcomes

- Verify that OpenShift Service Mesh (OSSM) defines the *permissive* mTLS mode as the default.
- Configure *strict* mTLS mode at the namespace level by using the peer authentication policy.
- Validate mTLS enforcement by testing encrypted and plain-text traffic.
- Visualize mTLS behavior and SPIFFE identities in Kiali.
- Observe automatic certificate rotation and verify zero-downtime transitions.

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshsecurity-mtls
```

Also, run the following command to update your `$PATH` variable and make the `traffic_gen.py` command available immediately. Run it once after creating a new environment.

```
[student@workstation ~]$ source ~/.bashrc
```

The `lab start` command performs the following activities:

- Creates the `mesh-outside` and the `meshsecurity-mtls` namespaces.
- Adds only the `meshsecurity-mtls` namespace to the service mesh.
- Deploys the `curl`, `ratings`, and `reviews-v2` applications inside the mesh at the `meshsecurity-mtls` namespace.
- Deploys the `curl` application outside the mesh at the `mesh-outside` namespace.

Instructions

1. Verify the initial status of the service mesh and the default mTLS configuration.
 - 1.1. In a new terminal window, log in to the OpenShift cluster as the developer user with the developer password, then switch to the `meshsecurity-mtls` project.

```
[student@workstation ~]$ oc login -u developer \
-p developer https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation ~]$ oc project meshsecurity-mtls
...output omitted...
```



- 1.2. Verify that the applications are running in the `meshsecurity-mtls` namespace.

```
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
curl-...	2/2	Running	0	1m
ratings-v1-...	2/2	Running	0	1m
reviews-v2-...	2/2	Running	0	1m

All pods should show 2/2 containers ready, indicating that Istio injected the Envoy sidecar proxy.

- 1.3. Verify that outside the mesh curl pod at the mesh-`outside` namespace is running without a sidecar.

```
[student@workstation meshsecurity-mtls]$ oc get pods -n mesh-outside
```

NAME	READY	STATUS	RESTARTS	AGE
curl-...	1/1	Running	0	2m

Notice that the curl pod should show 1/1 containers ready, indicating it does not have the Envoy sidecar proxy.

- 1.4. Change to the exercise directory.

```
[student@workstation ~]$ cd ~/course/labs/meshsecurity-mtls
[student@workstation meshsecurity-mtls]$
```

2. Verify the default mTLS configuration.

By default, OpenShift Service Mesh operates in *permissive* mode when no peer authentication policies exist. In this mode, services accept both mTLS-encrypted and plain-text traffic, providing flexibility during migration.

- 2.1. Verify that no peer authentication policy exists in the meshsecurity-mtls namespace.

```
[student@workstation meshsecurity-mtls]$ oc get peerauthentication
No resources found in meshsecurity-mtls namespace.
```

No peer authentication policy exist in the meshsecurity-mtls namespace.

- 2.2. Verify that no peer authentication policy exists in the istio-system namespace.

```
[student@workstation meshsecurity-mtls]$ oc get peerauthentication \
-n istio-system
No resources found in istio-system namespace.
```

No peer authentication policy exist in the istio-system namespace. With no peer authentication policy configured, OpenShift Service Mesh uses the default *permissive* mode.

In *permissive* mode, services accept both mTLS-encrypted and plain-text traffic.

- 2.3. Store the inside mesh curl pod name in the CURL_INSIDE_POD environment variable.

```
[student@workstation meshsecurity-mtls]$ CURL_INSIDE_POD=$( \
oc get pods -l app=curl \
-o jsonpath='{.items[*].metadata.name}')
```

- 2.4. Verify that internal mesh traffic works correctly. Use the inside mesh curl pod to make a request to the reviews service.

```
[student@workstation meshsecurity-mtls]$ oc exec $CURL_INSIDE_POD \
-- curl -s reviews.meshsecurity-mtls.svc.cluster.local:9080/reviews/1 | jq
{
  "id": "1",
  "podname": "reviews-v2-64df7bdcbf-542gt",
  "clustername": "null",
  "reviews": [
    {
      "reviewer": "Reviewer1",
      "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshin
g!",
      "rating": {
        "stars": 5,
        "color": "black"
      }
    },
    {
      "reviewer": "Reviewer2",
      "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to oth
er plays by Shakespeare.",
      "rating": {
        "stars": 4,
        "color": "black"
      }
    }
  ]
}
```

The request succeeds because both the inside mesh curl pod and reviews service have Envoy sidecars that establish mTLS connections automatically, even in *permissive* mode. Repeat the command 5-10 times to make it easier to visualize on the Kiali graph.

- 2.5. Store the outside mesh curl pod name in the `CURL_OUTSIDE_POD` environment variable.

```
[student@workstation meshsecurity-mtls]$ CURL_OUTSIDE_POD=$(
oc get pods -l app=curl -n mesh-outside \
-o jsonpath='{.items[*].metadata.name}')
```

- 2.6. Verify that plain-text traffic from outside the mesh also works in *permissive* mode.

```
[student@workstation meshsecurity-mtls]$ oc exec -n mesh-outside $CURL_OUTSIDE_POD \
-- curl -s reviews.meshsecurity-mtls.svc.cluster.local:9080/reviews/1 | jq
{
  "id": "1",
  "podname": "reviews-v2-64df7bdcbf-542gt",
  "clustername": "null",
  "reviews": [
    {
      "reviewer": "Reviewer1",
      "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshin
g!",
      "rating": {
        "stars": 5,
        "color": "black"
      }
    },
    {
      "reviewer": "Reviewer2",
      "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to oth
er plays by Shakespeare.",
      "rating": {
        "stars": 4,
        "color": "black"
      }
    }
  ]
}
```

The request succeeds, confirming that *permissive* mode accepts plain-text traffic. Repeat the command 5-10 times to make it easier to visualize on the Kiali graph.

- 2.7. In the workstation machine, open the OpenShift web console in your browser.

Go to <https://console-openshift-console.apps.ocp4.example.com>, click `htpasswd_provider` and use `developer` as the user and the password. If prompted about the guided tour, then click `Skip tour`.

- 2.8. In the **Administrator** perspective in the OpenShift console click **Service Mesh** → **Traffic Graph**.
- 2.9. Add the `meshsecurity-mtls` namespace to the graph by selecting it in **Select Namespaces**.
- 2.10. In the **Display** menu, enable the **Security** option.
- 2.11. Ensure that you have sufficient time range to see traffic data. Select **Traffic metrics per refresh** → **Last 1h**.

Observe the current traffic graph showing *permissive* mode behavior:

- *Inside mesh traffic* (`curl` → `reviews` → `ratings`): Shows padlock icons indicating mTLS encryption between services.
- *Outside mesh traffic* (`unknown` → `reviews`): Shows *no* padlock icon because the outside mesh `curl` pod sends plain-text traffic.

The unknown source corresponds to the `curl` pod in the `mesh-outside` namespace, which has no Envoy sidecar. This demonstrates that *permissive* mode accepts both encrypted (mTLS) and plain text traffic.

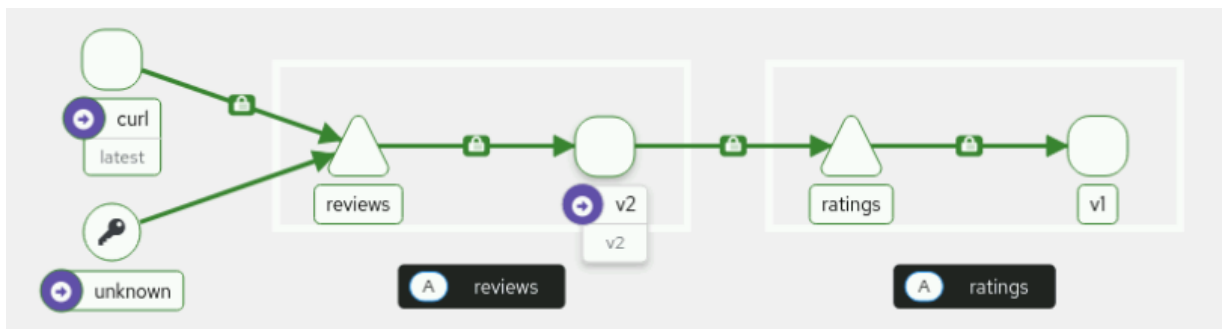


Figure 1.10:
Traffic graph with mTLS *permissive* mode

Leave this browser tab open. You return to this graph throughout the exercise.

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Generate more traffic by refreshing the page, re-running `traffic_gen.py`, or running the `curl` commands again.

3. Enforce use of mTLS in the `meshsecurity-mtls` namespace.

In this step, create a peer authentication policy to enforce *strict* mTLS mode for all workloads in the `meshsecurity-mtls` namespace. This ensures that only encrypted and mutually authenticated traffic is accepted.

In production environments, you would typically migrate from *permissive* to *strict* mode gradually to avoid service disruptions. The *permissive* mode enables you to verify that all services support mTLS before enforcing strict mode.

- 3.1. Review the `ns-mtls.yaml` file that sets the mTLS mode to strict.

```
[student@workstation meshsecurity-mtls]$ cat ns-mtls.yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT ❶
```

- ❶ The STRICT mode ensures that only mTLS traffic is accepted.

- 3.2. Create the peer authentication OSSM resource.

```
[student@workstation meshsecurity-mtls]$ oc create -f ns-mtls.yaml
peerauthentication.security.istio.io/default created
```

4. Verify *strict* mTLS enforcement and test access restrictions.

After applying *strict* mode, validate that the policy works as expected. Services with sidecars continue communicating via mTLS, whereas plain-text traffic is rejected.

4.1. Verify that the inside mesh `curl` pod can still make requests to the reviews service.

```
[student@workstation meshsecurity-mtls]$ oc exec $CURL_INSIDE_POD \
-- curl -s reviews.meshsecurity-mtls.svc.cluster.local:9080/reviews/1 | jq
{
  "id": "1",
  "podname": "reviews-v2-64df7bdcbf-zrr5q",
  "clustername": "null",
  "reviews": [
    {
      "reviewer": "Reviewer1",
      "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!",
      "rating": {
        "stars": 5,
        "color": "black"
      }
    },
    {
      "reviewer": "Reviewer2",
      "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.",
      "rating": {
        "stars": 4,
        "color": "black"
      }
    }
  ]
}
```

The request succeeds because both pods have sidecars that establish mTLS connections. Repeat the command 5-10 times to make it easier to visualize on the Kiali graph.

4.2. Verify that the outside mesh `curl` pod cannot make requests to the reviews service.

```
[student@workstation meshsecurity-mtls]$ oc exec -n mesh-outside $CURL_OUTSIDE_POD \
-- curl -s reviews.meshsecurity-mtls.svc.cluster.local:9080/reviews/1
command terminated with exit code 56
```

The request fails because the outside mesh `curl` pod cannot establish an mTLS connection. In *strict* mode, the reviews service only accepts mTLS traffic and rejects plain-text connections.

4.3. Inspect SPIFFE identities in the traffic graph.

Return to the browser tab showing the traffic graph in **Service Mesh** → **Traffic Graph**. You might adjust the **Traffic metrics per refresh** → **Last 5m**.

Click on the edge between `curl` and `reviews` to view the connection details. The sidebar shows the SPIFFE identities for the source and destination workloads:

- *Source identity*: `spiffe://cluster.local/ns/meshsecurity-mtls/sa/curl`
- *Destination identity*: `spiffe://cluster.local/ns/meshsecurity-mtls/sa/bookinfo-reviews`

These identities are automatically assigned based on the Kubernetes service accounts and are used for mutual authentication in mTLS connections.

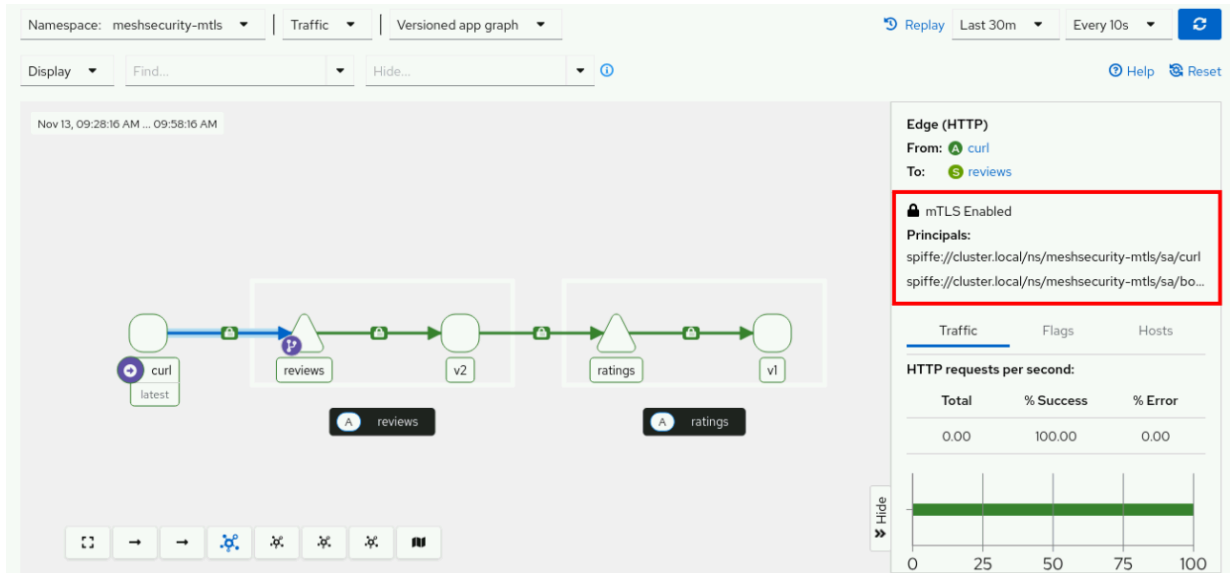


Figure 1.11: SPIFFE identities in the traffic graph

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Generate more traffic by refreshing the page, re-running `traffic_gen.py`, or running the `curl` commands again.

5. Observe the automatic certificate rotation process.

OpenShift Service Mesh automatically manages certificate lifecycle through Istiod. In this step, trigger certificate rotation by deleting the pod, which forces Envoy to request a new certificate from Istiod.

Automatic certificate rotation demonstrates the following characteristics:

- Zero-downtime certificate rotation
- Automatic certificate issuance by Istiod
- Unique certificate identities per pod
- Seamless mTLS continuation after rotation

5.1. Store the reviews pod name in the REVIEWS_POD environment variable.

```
[student@workstation meshsecurity-mtls]$ REVIEWS_POD=$(
oc get pods -l app=reviews \
-o jsonpath='{.items[*].metadata.name}')
```

5.2. Capture the current certificate serial number for the reviews pod.

```
[student@workstation meshsecurity-mtls]$ istioctl proxy-config \
secret $REVIEWS_POD | grep -B 1 default
RESOURCE NAME  TYPE      STATUS  VALID  CERT  SERIAL NUMBER  ...
default        Cert Chain ACTIVE  true   6756ffa088efaba437bd7b2602f743f3  ...
```

Your values might differ.

Note the serial number. This uniquely identifies the current certificate.

5.3. Delete the reviews pod to trigger certificate rotation.

```
[student@workstation meshsecurity-mtls]$ oc delete pod $REVIEWS_POD
pod "reviews-v2-..." deleted
```

The deployment controller automatically creates a new pod to replace it.

5.4. Wait for the new pod to be ready and update the REVIEWS_POD environment variable.

```
[student@workstation meshsecurity-mtls]$ oc wait --for=condition=Ready \
pod -l app=reviews --timeout=60s
pod/reviews-v2-... condition met

[student@workstation meshsecurity-mtls]$ REVIEWS_POD=$(
oc get pods -l app=reviews \
-o jsonpath='{.items[*].metadata.name}')

```

5.5. Verify that the new pod received a new certificate with a different serial number.

```
[student@workstation meshsecurity-mtls]$ istioctl proxy-config \
secret $REVIEWS_POD | grep -B 1 default
RESOURCE NAME  TYPE      STATUS  VALID CERT  SERIAL NUMBER  ...
default        Cert Chain ACTIVE   true         4fa5fe7e69559b5840e2bcad9e8e9597  ...

```

Your values might differ.

The serial number differs from the previous certificate, confirming that Istiod automatically issued a new certificate. The new certificate has updated Not Before and Not After timestamps.

5.6. Verify that mTLS communication continues to work with the new certificate.

```
[student@workstation meshsecurity-mtls]$ oc exec $CURL_INSIDE_POD \
-- curl -s reviews.meshsecurity-mtls.svc.cluster.local:9080/reviews/1 | jq
{
  "id": "1",
  "podname": "reviews-v2-64df7bdcbf-rqkn9",
  "clustername": "null",
  "reviews": [
    {
      "reviewer": "Reviewer1",
      "text": "An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!",
      "rating": {
        "stars": 5,
        "color": "black"
      }
    },
    {
      "reviewer": "Reviewer2",
      "text": "Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare.",
      "rating": {
        "stars": 4,
        "color": "black"
      }
    }
  ]
}

```

The request succeeds, demonstrating that certificate rotation happens seamlessly without service disruption.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshsecurity-mtls

```

Revision: ad0009I-3.1-ed3-20260112-486e433



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트
(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책 (<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침 (<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정